



Research Diary

Can neural networks reliably estimate kerning, and how much does context matter?

Lab: Computational Intelligence Lab (CILab)

Supervisor: Dr. Gennaro Vessio

Project: Learning to Kern: Comparison of Architectures for Optimal Letter Space

Collaborators:

- Angelantonio Fedele Murolo – M.Sc. student
- Emanuele Colonna – Ph.D. student

Start date: Wednesday 10th June, 2026

End date: Wednesday 2th September, 2026

Saturday 5th September, 2026

Contents

1	Introduction	2
2	Related Work	3
3	Dataset	3
4	Models	5
4.1	Pairwise model	7
4.2	Set-wise model	7
4.3	Context-augmented hybrids	8
5	Experiments and Results	8
5.1	Setup	8
5.2	Main results	10
5.3	Style and per-font behavior	11
6	Analysis	13
7	Interactive Demo	18
8	Conclusion and Future Work	19

Abstract

Kerning—the adjustment of the horizontal space between pairs of letters—is still performed manually or with imperfect heuristics. We replicate and extend *Learning to Kern* [7], which frames the task as supervised regression over all $52 \times 52 = 2,704$ ordered pairs of Latin letters. We build a corpus of 7,263 proportional Google Fonts with 19,639,152 targets and evaluate five architectures on a frozen ResNet18 backbone: the pairwise and set-wise models of the reference study, an ensemble, a set-to-pair fusion model, and a cross-attention model. The pairwise model is the strongest single architecture (3.89 px test MAE, 70.7% below the reference pairwise result and 3.2% of the 122.6-px mean spacing); the ensemble is marginally better (3.88 px); the set-wise Transformer (4.67 px) and the hybrids do not surpass this baseline. SHAP analysis shows the pairwise model relies on glyph shape (95.9% of feature importance) rather than letter identity, and near-uniform attention maps indicate that a compact single-layer Transformer cannot exploit global context over 2,704 tokens, while hidden states frozen for one objective do not transfer to another. The study isolates the factors governing accuracy in this regime: a rendering-consistent all-pairs target, corpus scale, and the transferability of frozen representations.

Keywords: kerning · letter spacing · deep learning · font · letter

Introduction

Kerning is the micro-typographic task of adjusting the horizontal space between adjacent letters in proportional typefaces. For the 52 letters of the Latin alphabet, a designer must specify $52 \times 52 = 2,704$ pair-specific spacings per font, since the ideal distance depends on the shapes involved (the diagonal strokes of A and V invite tighter spacing than the parallel stems of H and H) and on the style of the typeface. In practice kerning is set manually or with heuristic rules such as optical spacing [4], refined in professional systems like URW’s hz-program [3] and packaged in FontForge’s automatic mode [8]; these heuristics operate on local geometry only and still require expert revision. Nakatsuru and Uchida [7] formalized the task as supervised learning, training on the manually tuned spacing of existing fonts a *pairwise* model—regressing one spacing at a time from two glyph representations—and a *set-wise* Transformer that predicts all N^2 spacings of a font jointly so that self-attention can enforce global consistency. On 2,558 fonts they reported a set-wise advantage (5.32 vs. 13.27 px MAE).

This work replicates that framework on a substantially larger corpus and probes its central claim. We contribute:

- **A larger, denser corpus.** We grow the dataset to 7,263 fonts and define the regression target for *all* N^2 ordered pairs, treating absent kerning entries as zero, exactly as font rendering does (Section 3).
- **A systematic architectural comparison.** Beyond the two reference models, we evaluate an ensemble, a set-to-pair fusion model, and a cross-attention model that injects set-wise context into the pairwise formulation (Section 4).
- **A mechanistic explanation of the ranking.** SHAP, Grad-CAM, and attention diagnostics explain why the pairwise formulation wins at this scale (Section 6).

The main finding reverses the reference conclusion: with a dense all-pairs target and a larger corpus, the pairwise model outperforms the set-wise Transformer, and context-injecting hybrids do not close the gap.

Related Work

Typographic practice treats spacing as a set-level property of a typeface: reference pairs are spaced first and the remaining pairs are adjusted relative to them [4]. Automatic approaches predate machine learning: optical spacing estimates the blank area between facing contours [4], the hz-program extended micro-typographic adjustment to whole text strings [3], and FontForge packages a heuristic automatic kerning mode widely used as a first pass [8]. These methods use only the local geometry of the two glyphs and still need expert revision.

Nakatsuru and Uchida [7] introduced the first comprehensive deep-learning framework for kerning. Their pairwise model concatenates two frozen ResNet18 [2] features with one-hot identity vectors and regresses a scalar spacing through a small DNN; their set-wise model builds N^2 pair tokens per font and processes them with a Transformer encoder [11], so that every pair estimate can attend to every other pair of the same font. On 2,558 Google Fonts the set-wise model clearly dominated, an advantage the authors attributed to globally consistent kerning. Macro-typographic tasks such as layout generation have been widely studied with generative models and Transformers [11], but micro-typography had received little attention before that work.

Our study is a scaled replication of the reference framework with an extended model family. The explainability tools used to diagnose the models are SHAP [6], Grad-CAM [9], and t-SNE [10]; the optimizer is Adam [5].

Dataset

Let a font be represented by $N = 52$ glyphs corresponding to the Latin letters A–Z and a–z. For an ordered pair (L, R) , the ground-truth horizontal spacing is

$$t(L, R) = \text{advance}(L) + k(L, R) \cdot s + c_x(R) - c_x(L), \quad (1)$$

where $\text{advance}(L)$ is the advance width of the left glyph, $k(L, R)$ is the explicit kerning adjustment in font units, s is the per-font scale, and $c_x(\cdot)$ is the horizontal centroid of a glyph. Because renderers treat an absent kerning entry as no adjustment, we set $k(L, R) = 0$ whenever the font declares no entry for the pair; the target is therefore defined for every ordered pair, and models minimize the mean absolute error (MAE) in pixels over all pairs.

The corpus is drawn from Google Fonts [1]: after removing monospaced families, where spacing is trivial by construction, 1,868 proportional families (7,284 TTF variants) are downloaded without failures. Ground-truth kerning is extracted with fontTools from GPOS PairPos tables, with the legacy **kern** table as fallback, yielding 715,826 nonzero entries—a rate of 2.7% over all declared pairs, with values from -741 to $+696$ font units (Figure 1, top). Each font is rasterized with FreeType into 52 binary 256×256 images sharing a baseline at $y = 160$, with a per-font scale found by binary search (Figure 2); 7,263 fonts are accepted (21 rejected for invalid shared scale or rendering failures) and the N^2 targets of Equation 1 are computed for each, giving 19,639,152 targets overall (Figure 1, bottom; mean 122.6 px, range -31.7 to 303.0). Targets are addressed lazily and never materialized: the dataloaders store one cumulative offset per font and resolve a global pair index to a font and a local pair by binary search and modular arithmetic. Table 1 summarizes the corpus. Splits are defined at the font-family level with a fixed seed so that no family spans two partitions: 1,488 families (5,758 fonts) for training, 186 (719) for validation, and 187 (786) for test, for 15,569,632/1,944,176/2,125,344 targets. The test set spans

the four Google Fonts style categories (Sans-serif 463, Serif 167, Display 80, Handwriting 76).

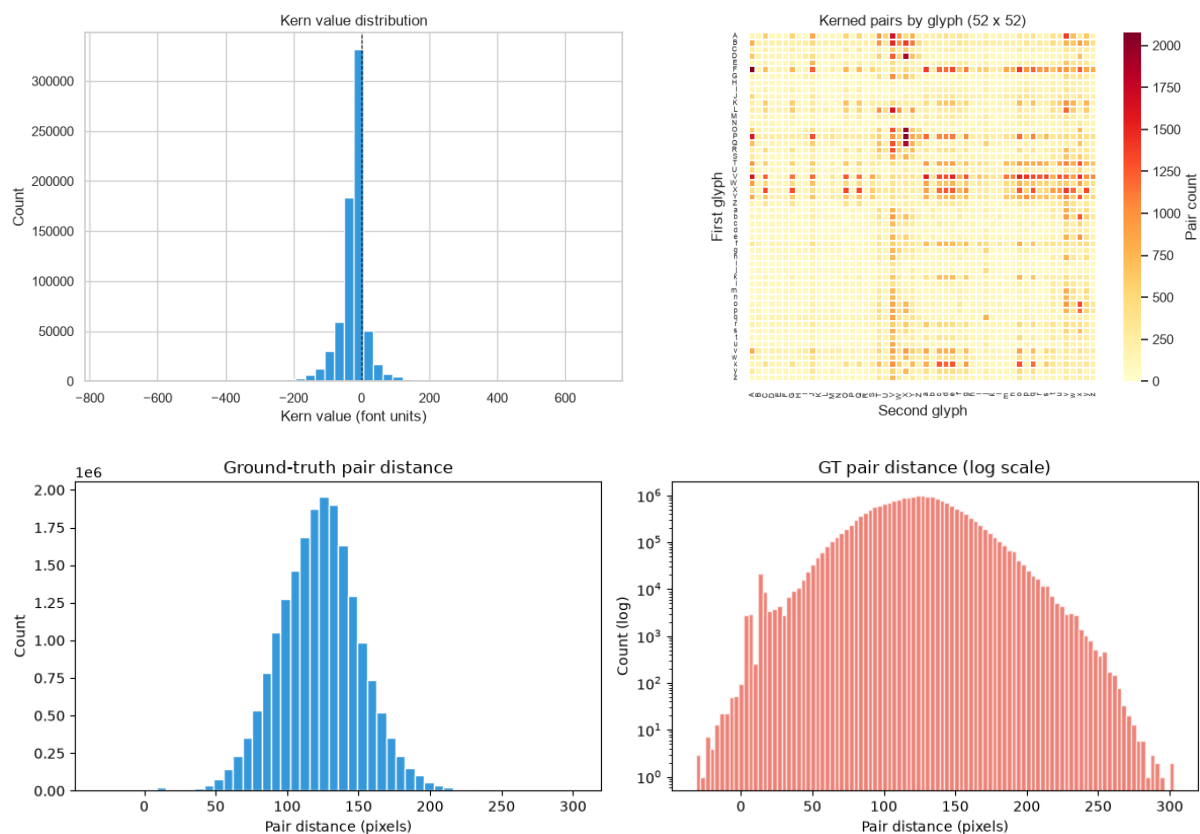


Figure 1: Top: kerning values in font units (nonzero entries only) and the 52×52 count matrix of explicitly kerned pairs. Bottom: distribution of the ground-truth pair distances across the corpus (linear and logarithmic scale).

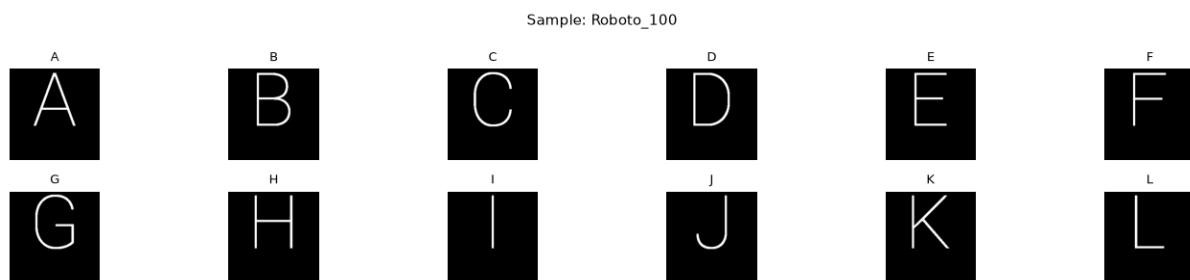


Figure 2: Rasterized glyphs for a representative font (Roboto Thin): the first twelve glyphs on the shared 256×256 canvas with baseline at $y = 160$.

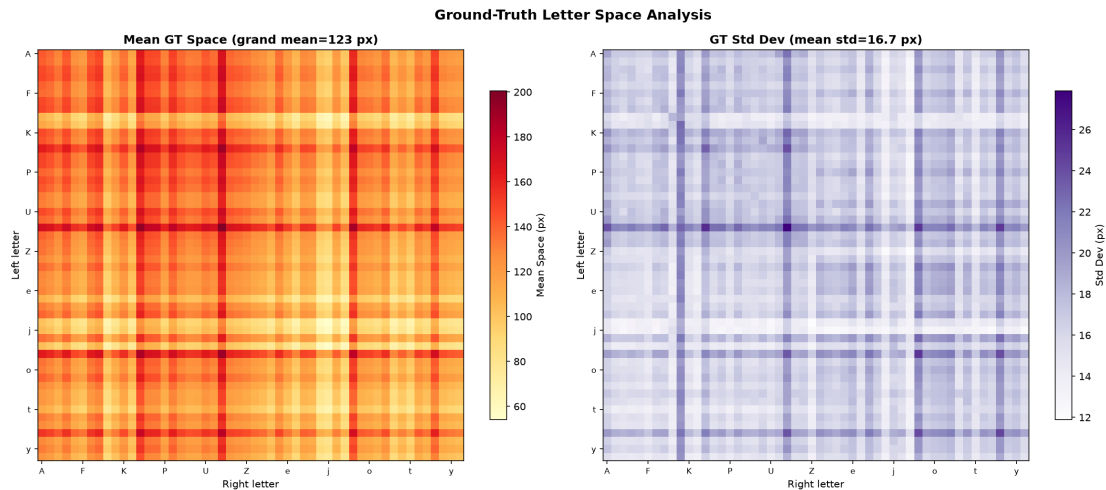


Figure 3: Mean (left) and standard deviation (right) of the ground-truth spacing over the 52×52 pairs across the test fonts. The high-variance pairs—dominated by W—are where no per-pair constant can succeed.

Table 1: Corpus statistics.

Quantity	Value
Proportional families (after monospace filter)	1,868
Downloaded TTF variants	7,284
Valid fonts after rasterization	7,263
Glyph images (52 per font)	377,676
All-pairs targets	19,639,152
Explicit (kern $\neq 0$) pairs	715,826
Nonzero-kern rate over declared pairs	2.7%
Mean ground-truth spacing	122.6 px

Table 2: Family-level split (fixed seed).

Split	Families	Fonts	Pairs
Training	1,488	5,758	15,569,632
Validation	186	719	1,944,176
Test	187	786	2,125,344
Total	1,861	7,263	19,639,152

Models

All models share a frozen feature backbone and differ in how they consume it; Figure 4 gives an overview.

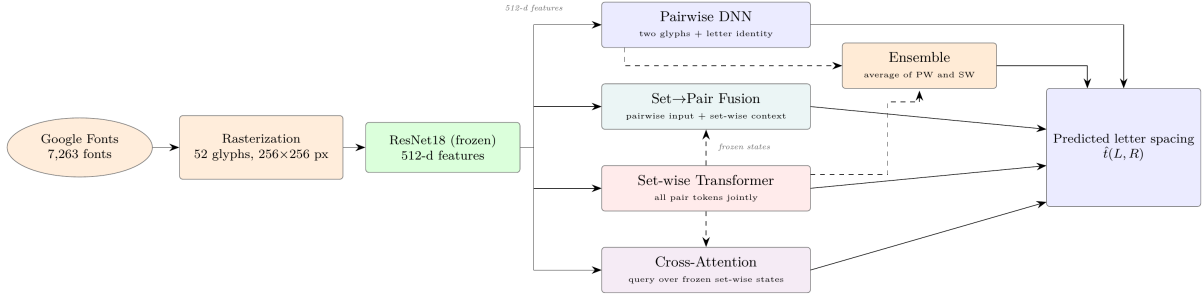


Figure 4: Overview of the pipeline. Left to right: the Google Fonts corpus, rasterization, and the frozen ResNet18 backbone. The 512-d features feed the four trained estimation heads, while the ensemble combines the pairwise and set-wise predictions. Dashed arrows denote the frozen set-wise hidden states consumed by the two hybrids and the averaging of the two base predictions.

Each glyph is encoded by a ResNet18 [2] trained from scratch on 52-way glyph classification. The single-channel image is replicated across three channels and normalized to $[-1, 1]$; the network (11,203,188 parameters) is trained with cross-entropy and Adam [5] with early stopping (patience 5). The best validation loss occurs at epoch 3; training stops at epoch 8 with 0.980 validation and 0.971 test accuracy, and the final-epoch weights are used (Figure 5). Removing the classification head leaves a frozen trunk that maps every glyph to a 512-dimensional feature, cached per font. These features are shared by all downstream heads, so their quality is a hard upper bound on everything that follows; a t-SNE projection [10] confirms that the space is morphologically organized, with uppercase and lowercase letters in coherent clusters and visually similar letters (e.g., 0/Q) close together (Figure 6).

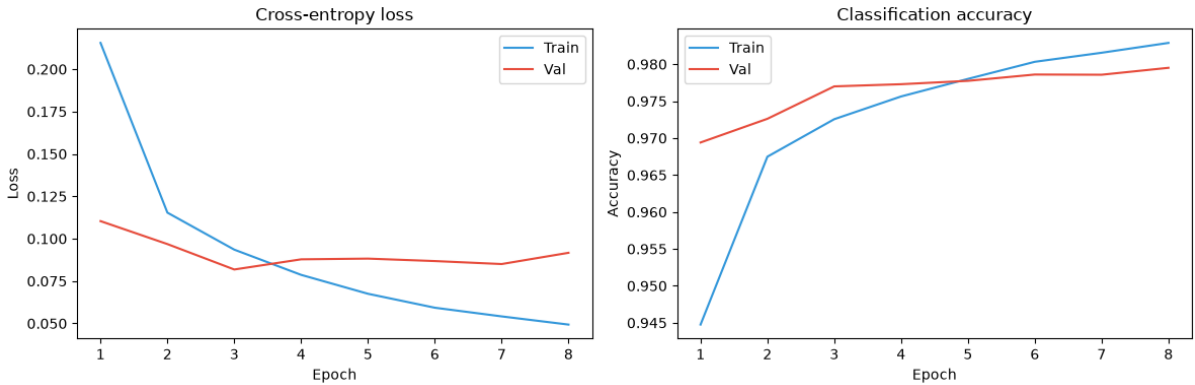


Figure 5: ResNet18 pre-training: cross-entropy loss and classification accuracy. Training stops at epoch 8 under early stopping.

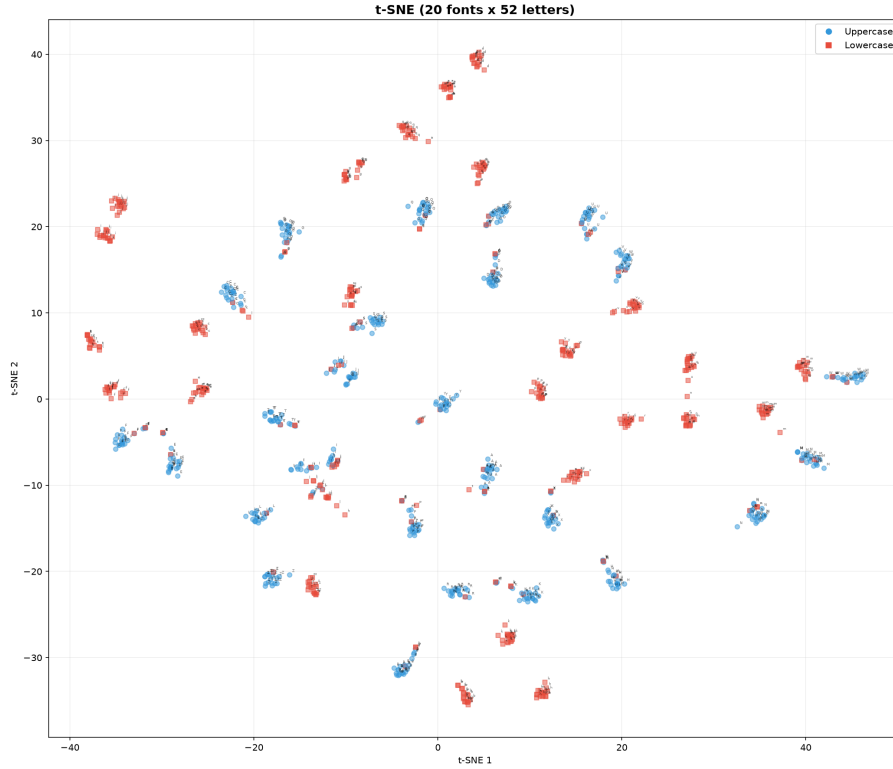


Figure 6: t-SNE projection of the ResNet18 feature space for the 52 letters across 20 sampled fonts. Uppercase and lowercase glyphs form coherent clusters.

Pairwise model

The pairwise model estimates the spacing of one ordered pair from its two glyph representations. Its input concatenates the left glyph’s 512-d feature f_L , the right glyph’s 512-d feature f_R , and two 52-d one-hot identity vectors,

$$x = [f_L; f_R; \text{oh}_L; \text{oh}_R] \in R^{1128}, \quad (2)$$

where the concatenation order encodes the direction of the pair, so that (A, V) and (V, A) remain distinguishable. Conditioning on the letter identities lets a single network serve all 2,704 pair classes—instead of one regressor per pair—with the one-hot vectors acting as a learned per-pair bias; Section 6 shows that this identity channel carries only a small fraction of the predictive signal. The predictor is an MLP with three linear layers ($1128 \rightarrow 256 \rightarrow 256 \rightarrow 1$, ReLU, 355,073 parameters). It is trained with the L1 loss—which regresses the evaluation metric directly and is robust to the heavy right tail of the targets—Adam at 10^{-4} , and batches of 64 pairs for up to 100 epochs (patience 20).

Set-wise model

The set-wise model operates on one font at a time and is expected to enforce global consistency: all 2,704 ordered pairs become tokens $t_{ij} = [f_i; f_j] \in R^{1024}$, distinct from t_{ji} , and a Transformer encoder [11] contextualizes the whole sequence so that each pair estimate can attend to every other pair of the font. Tokens carry no explicit identity and no positional encoding: the

input is an unordered set of pair representations, and the encoder is permutation-equivariant by construction. A linear projection first compresses each token to $d_{model} = 32$ —the main information bottleneck—then a single encoder layer (two heads, feed-forward dimension 128) processes the set, a final layer normalization produces the per-token states, and one weight-shared linear head maps every state to a scalar spacing (45,601 parameters total). The architecture follows the reference implementation [7]: one layer and $d_{model} = 32$, which Section 6 shows to be the limiting factor. Training uses the L1 loss summed over a font’s 2,704 predictions and normalized per pair, Adam at 10^{-3} , and batches of 64 fonts.

Context-augmented hybrids

The two hybrids test whether the set-wise model carries complementary information worth injecting into the pairwise formulation. Both reuse the trained set-wise model as a frozen feature source: its checkpoint is loaded without the output projection (`strict=False`), and the post-normalization hidden states of the 2,704 tokens are kept—the same states that the linear head of the set-wise model consumes—while only the new head is trained.

Set-to-pair fusion. The context is a single 32-d vector, the mean of the frozen hidden states over all tokens, appended to the pairwise input (1,160 dimensions total) and processed by the same MLP as the pairwise model (363,265 parameters). Mean pooling is the least expressive form of retrieval: it equals cross-attention with uniform weights, discarding any pair-specific structure.

Cross-attention. The pairwise encoder maps the 1,128-d input through $1128 \rightarrow 64 \rightarrow 64 \rightarrow 64$ into a query, which attends over the frozen hidden states with learned projections (Q : $64 \rightarrow 64$; K, V : $32 \rightarrow 64$; scaling by $1/\sqrt{64}$). The attended context is concatenated with the query and mapped by a $128 \rightarrow 256 \rightarrow 1$ head (122,241 parameters), so each pair retrieves a content-based context rather than the shared average used by the fusion model. Both hybrids are trained exactly like the pairwise model.

Ensemble. The ensemble averages the pairwise and set-wise predictions, $\hat{t}_{ENS} = (\hat{t}_{PW} + \hat{t}_{SW})/2$, with no trained weights: it combines a local-shape model and a set-context model with different inductive biases.

Experiments and Results

Setup

The primary metric is the flat test MAE over the 2,125,344 test pairs ($786 \text{ fonts} \times 2,704$); secondary views are the per-font MAE distribution, cumulative error distributions, and per-font wins against the pairwise model. Table 3 summarizes the training of the four heads (the backbone uses cross-entropy, Section 4); Figure 7 shows their training curves, which converge without a substantial generalization gap.

Table 3: Training configuration of the four trained heads (L1 loss, Adam, early-stopping patience 20, family-level split of Table 2, seed 42).

Model	Learning rate	Batch	Epochs run
Pairwise	10^{-4}	64 pairs	100
Set-wise	10^{-3}	64 fonts	100
Set-to-Pair Fusion	10^{-4}	64 pairs	65
Cross-Attention	10^{-4}	64 pairs	62

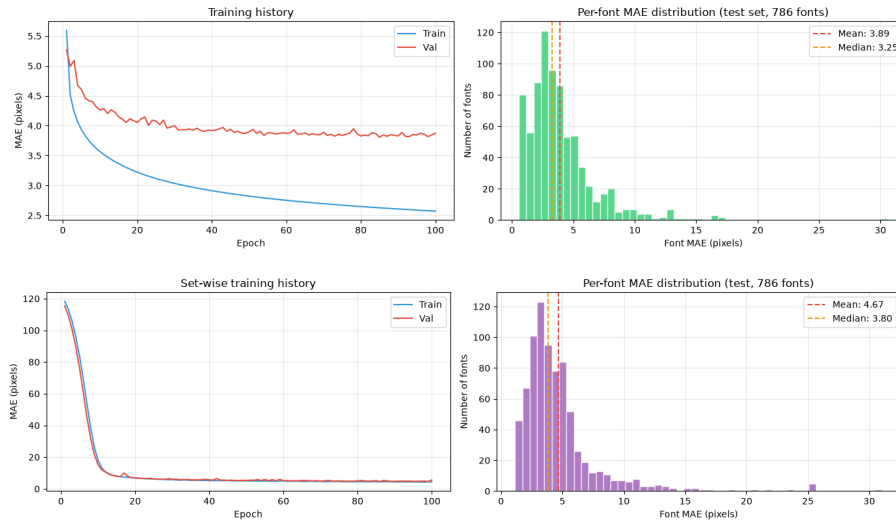


Figure 7: Training and validation MAE of the pairwise model (top) and the set-wise model (bottom), with the per-font test MAE distribution on the right.

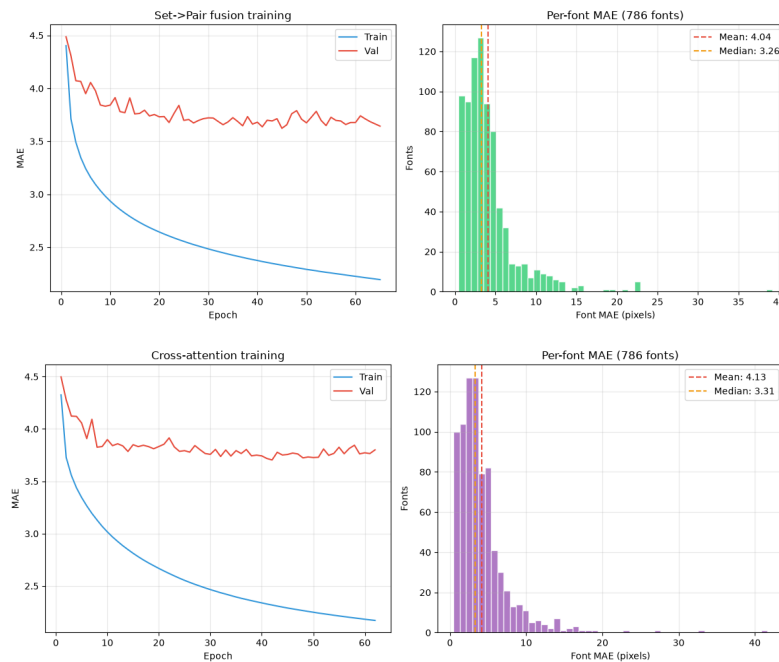


Figure 8: Training and validation MAE of the set-to-pair fusion model (top) and the cross-attention model (bottom). Both stop early under patience 20.

Main results

Table 4 reports the final results. The pairwise model is the strongest single architecture at 3.8949 px, and the training-free ensemble is best overall at 3.8804 px—but by only 0.0145 px over the pairwise model, a margin concentrated on the most difficult fonts. The two hybrids land between the pairwise and set-wise models, and the set-wise Transformer is last at 4.6705 px despite its global modeling assumptions. Figure 9 compares the per-font distributions of the pairwise, set-wise, and ensemble models; Figure 10 shows that the pairwise and ensemble curves dominate the set-wise curve at every error threshold (73.8% vs. 67.1% of pairs within 5 px; 92.1% vs. 89.4% within 10), confirming the ranking is a consistent ordering rather than an artifact of outliers.

Table 4: Final results on the 786 test fonts and all 2,704 pairs per font. The ensemble is training-free and has no validation history.

Model	Test MAE (px)	Best val (px)	Epochs	Parameters
Ensemble (PW+SW)/2	3.8804	—	—	—
Pairwise	3.8949	3.8063	100	355,073
Set-to-Pair Fusion	4.0446	3.6226	65	363,265
Cross-Attention	4.1320	3.7039	62	122,241
Set-wise	4.6705	4.6803	100	45,601

For scale, the reference study [7] reports a pairwise MAE of 13.27 px and a set-wise MAE of 5.32 px on its 2,558-font corpus, with the FontForge heuristic at 9.86 px; its average spacing of approximately 115 px is close to our 122.6 px, so the units are comparable. Against those anchors our pairwise model improves on the reference pairwise result by 70.7%, and our set-wise model on the reference set-wise result by 12.2%; the reference baselines were not re-executed on our corpus, so these cross-study comparisons are indicative rather than controlled. We return to this caveat in the Conclusion.

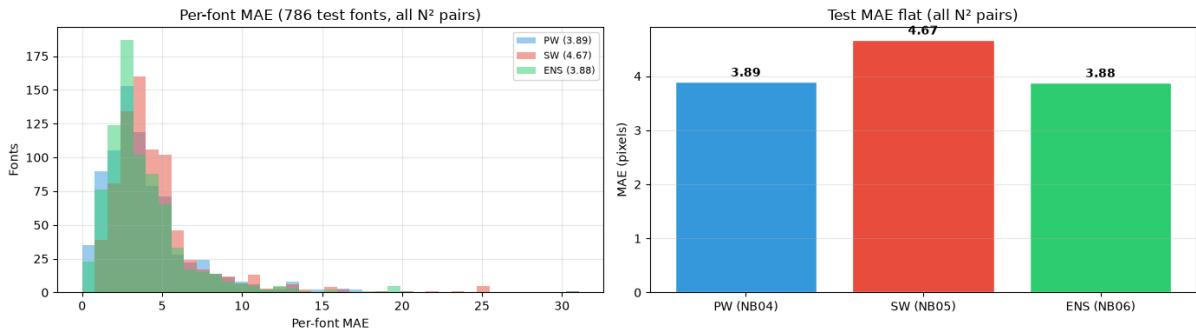


Figure 9: Per-font MAE distributions (left) and flat test MAE (right) of the pairwise, set-wise, and ensemble models. The ensemble is the sharpest, but its advantage over the pairwise model is only 0.0145 px.

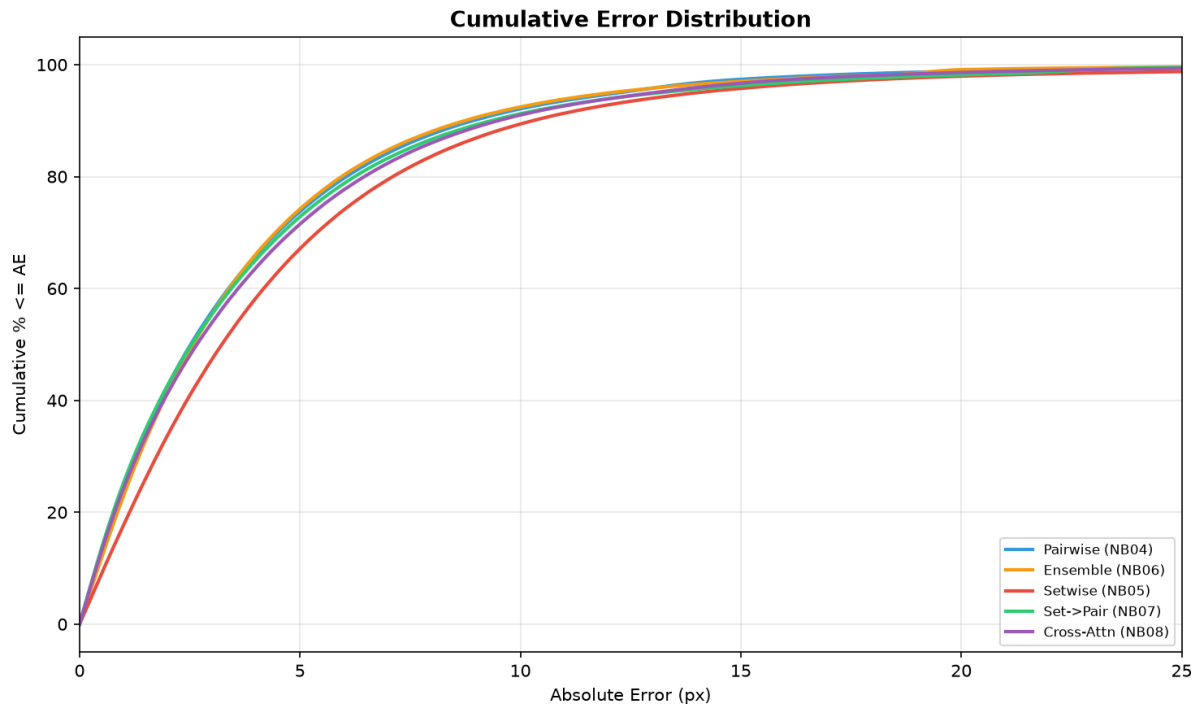


Figure 10: Cumulative error distributions of the five models. The pairwise and ensemble curves dominate at every threshold.

Style and per-font behavior

Table 5 breaks the MAE down by font style. Two regularities stand out. First, difficulty is stable across models: Sans-serif and Serif fonts are the easiest (3.44 and 3.26 px for the pairwise model), Display and Handwriting roughly twice as hard. Second, the model ranking is not fully style-invariant: the ensemble is best on Display (6.35 px) and the fusion model is the only architecture that beats the pairwise model on any style, winning on Handwriting (4.60 vs. 4.93 px).

Table 5: Per-style test MAE of the five evaluated models. Best model per row in bold.

Style	n	PW	SW	ENS	SP	CA
Sans-serif	463	3.44	4.17	3.48	3.79	3.78
Serif	167	3.26	4.13	3.32	3.36	3.45
Display	80	6.83	7.13	6.35	6.43	6.81
Handwriting	76	4.93	6.30	4.95	4.60	4.94

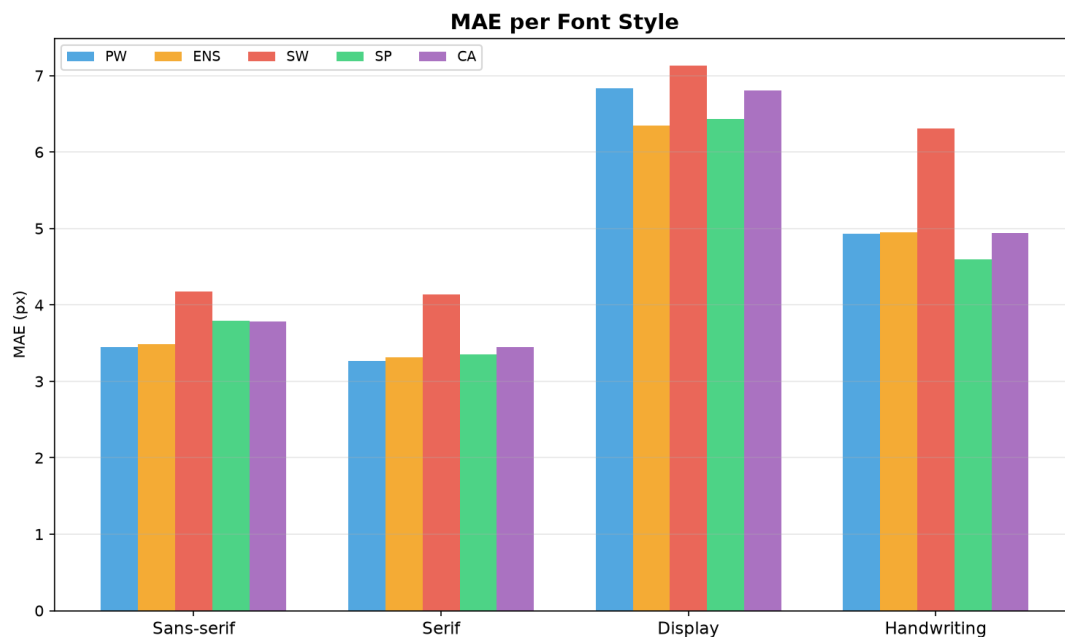


Figure 11: Per-style MAE across the five models. The ensemble lowers the error most on Display, the fusion model on Handwriting.

The global means hide where the models differ, so Table 6 counts per-font wins against the pairwise baseline. All alternatives win on roughly half of the fonts (406–407 of 786, 51.7–51.8%) except the set-wise model (340, 43.3%). The near-parity of the win counts with a worse global MAE for the hybrids is explained by *where* the wins happen: Figure 12 shows that the ensemble’s and fusion model’s victories concentrate on high-error fonts, while their regressions elsewhere are larger.

Table 6: Head-to-head wins against the pairwise model over the 786 test fonts.

Model	Fonts with lower MAE than PW	Share
Ensemble (PW+SW)/2	406	51.7%
Set-to-Pair Fusion	407	51.8%
Cross-Attention	407	51.8%
Set-wise	340	43.3%

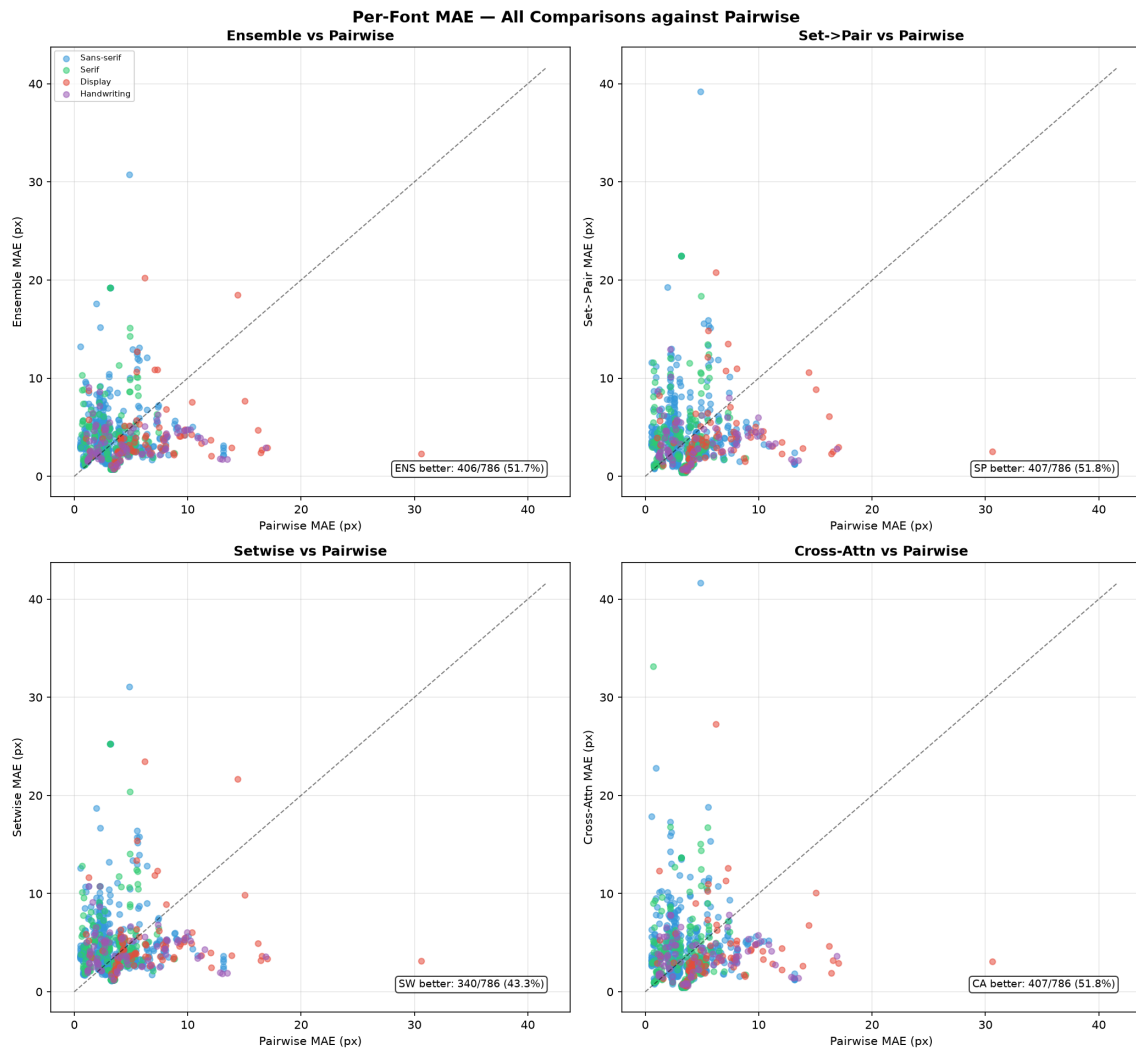


Figure 12: Per-font MAE of the ensemble and the hybrids against the pairwise model, colored by font style. Points below the diagonal are fonts where the alternative wins.

Analysis

The remaining question is *why* the ranking turns out this way. Figure 13 maps the per-pair MAE of each model onto the 52×52 grid, with a pairwise-minus-set-wise difference panel: the difference map is dominated by blue, so the pairwise advantage holds across most letter pairs, not merely on average. Error concentrates on pairs involving letters with diagonal or projecting strokes (W, V, T, lowercase f)—a property of the task itself, since those are the pairs whose optimal spacing varies most across fonts (Figure 3).

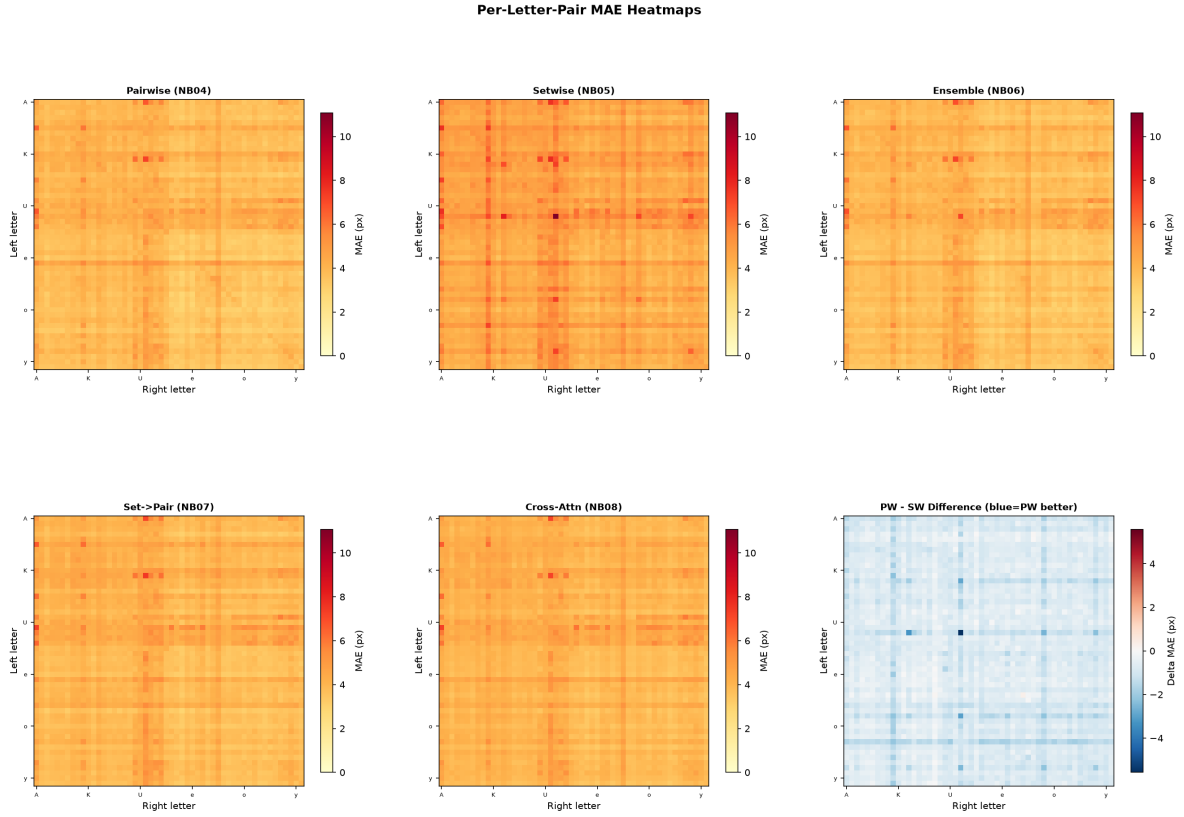


Figure 13: Per-letter-pair MAE heatmaps for the five models and the pairwise-minus-set-wise difference map (bottom right).

Zooming into individual pairs (Figure 14), the easy pairs of each model produce tight, low-error distributions centered near zero, while the hard pairs are wider and right-skewed, with a long tail of fonts where the model misses badly. The hard pairs are again dominated by characters with diagonal or projecting strokes, consistent with the letter ranking.

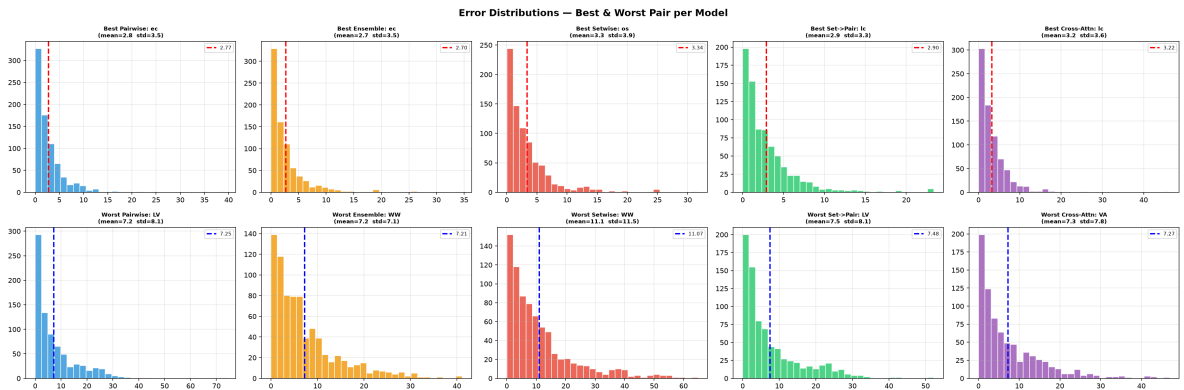


Figure 14: Error distributions for the best and worst letter pair of each model. Easy pairs (round-round combinations) yield tight distributions; hard pairs (typically involving W, V, or T) are wider and right-skewed.

The attention diagnostics make the mechanism concrete. The set-wise self-attention maps are close to uniform for the best, median, and worst test fonts (Figure 15, top): with one layer and $d_{model} = 32$, the model does not form the selective patterns that would let individual pairs

consult specific others. The cross-attention model behaves the same way (bottom): its distributions over the frozen states are nearly indistinguishable from uniform, with KL divergence in the range 0.88–1.09 and peak weights of only 18–30 $\times$ the uniform mass, so different pair queries do not lead to different attention profiles. These observations support two conclusions. First, the set-wise underperformance is a capacity statement, not a disproof of the idea: one 32-d layer over 2,704 tokens is too small to recover global structure, especially after the aggressive 1,024 $\rightarrow$ 32 projection. Second, the failure of both hybrids shows that representations frozen for one objective are not automatically queryable by another: the set-wise states were optimized for the set-wise scalar loss, and neither mean pooling nor cross-attention could extract a transferable signal from them.

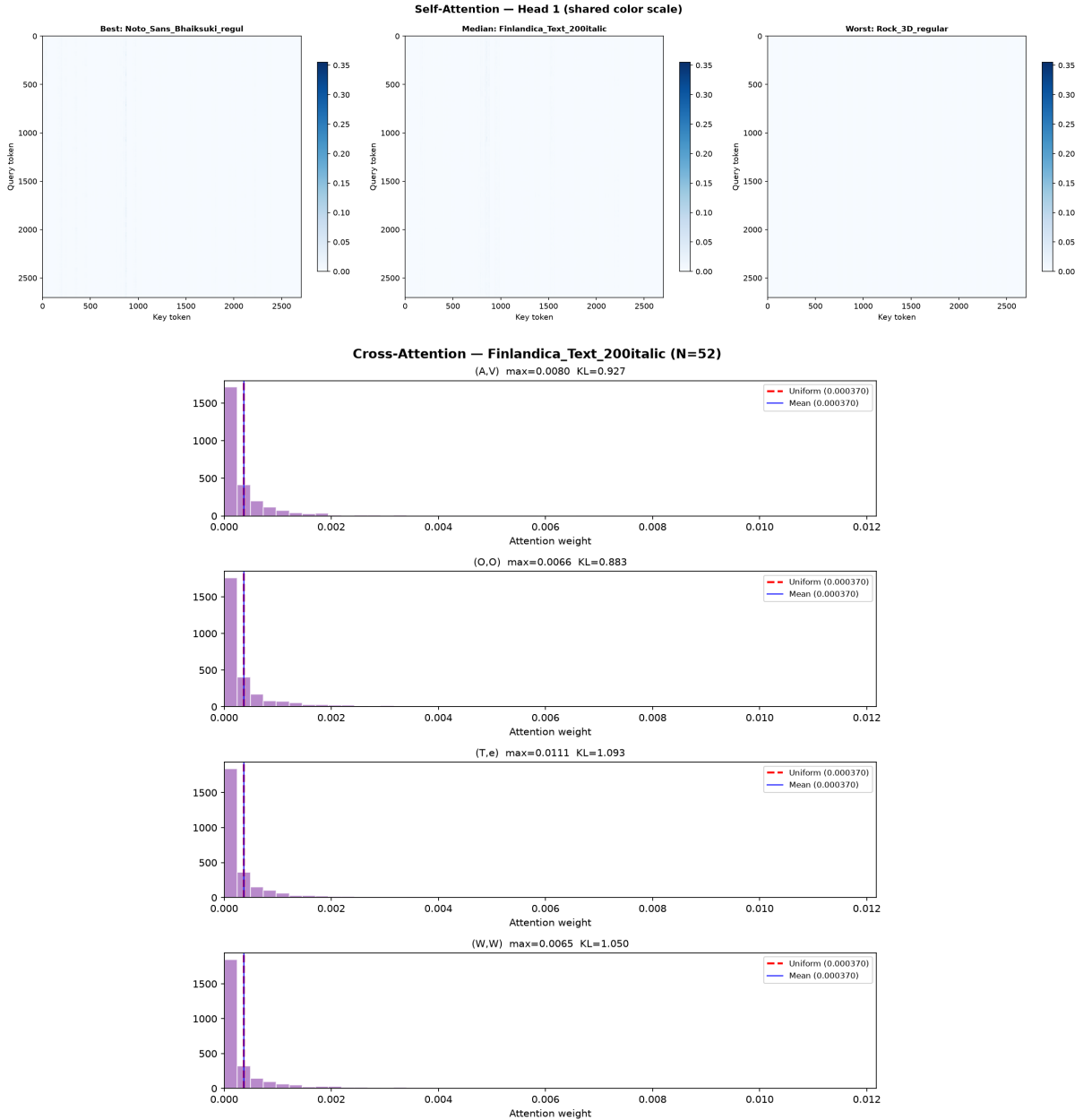


Figure 15: Top: set-wise self-attention maps for the best, median, and worst test fonts, close to uniform. Bottom: cross-attention weight distributions for representative pairs, close to the uniform baseline.

From the winner’s side, SHAP [6] decomposes the pairwise predictions over the 1,128-d input, grouped into visual features (2×512) and letter identity (2×52): the visual group accounts for 95.9% of the total importance and identity for only 4.1% (Figure 16)—the model spaces letters by their shape, not by which letters they are. Grad-CAM [9] on the backbone (Figure 17) shows the network attending to glyph edges and full silhouettes, exactly the regions that determine horizontal extent. The per-letter ranking (Figure 18) closes the analysis: the five models agree on which letters are hard (W, V, T, f), confirming that difficulty is a property of the task, while their absolute errors differ.

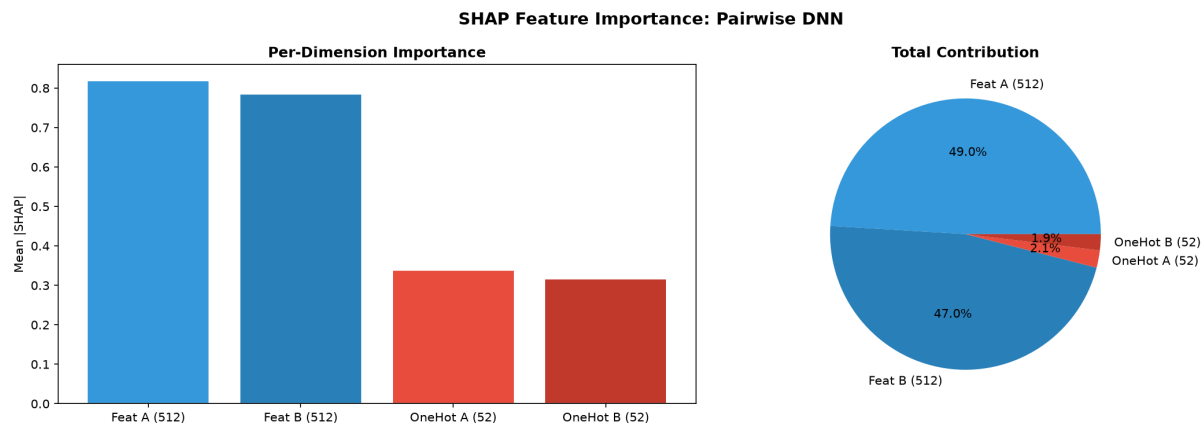


Figure 16: SHAP feature importance for the pairwise model. Visual features dominate letter identity by a wide margin (95.9% vs. 4.1%).

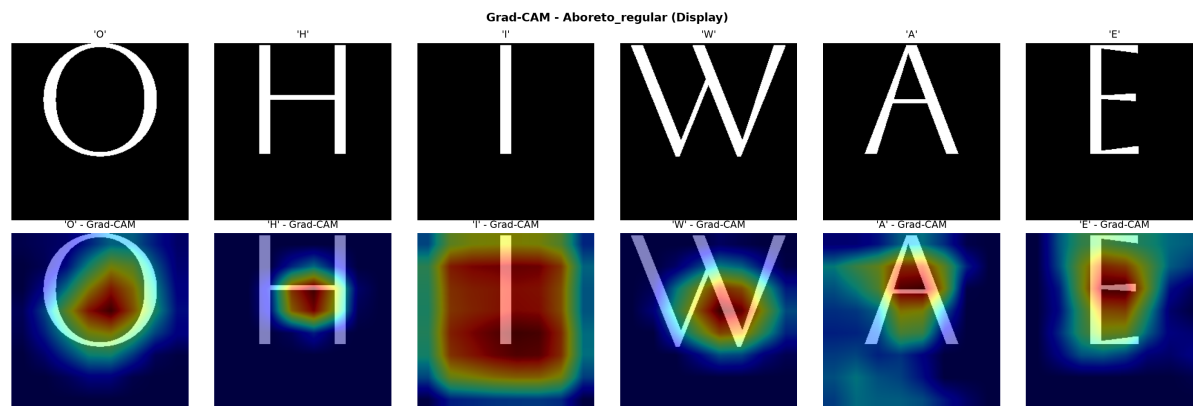


Figure 17: Grad-CAM visualizations for selected uppercase glyphs: the backbone concentrates on contours, crossbars, and stems.

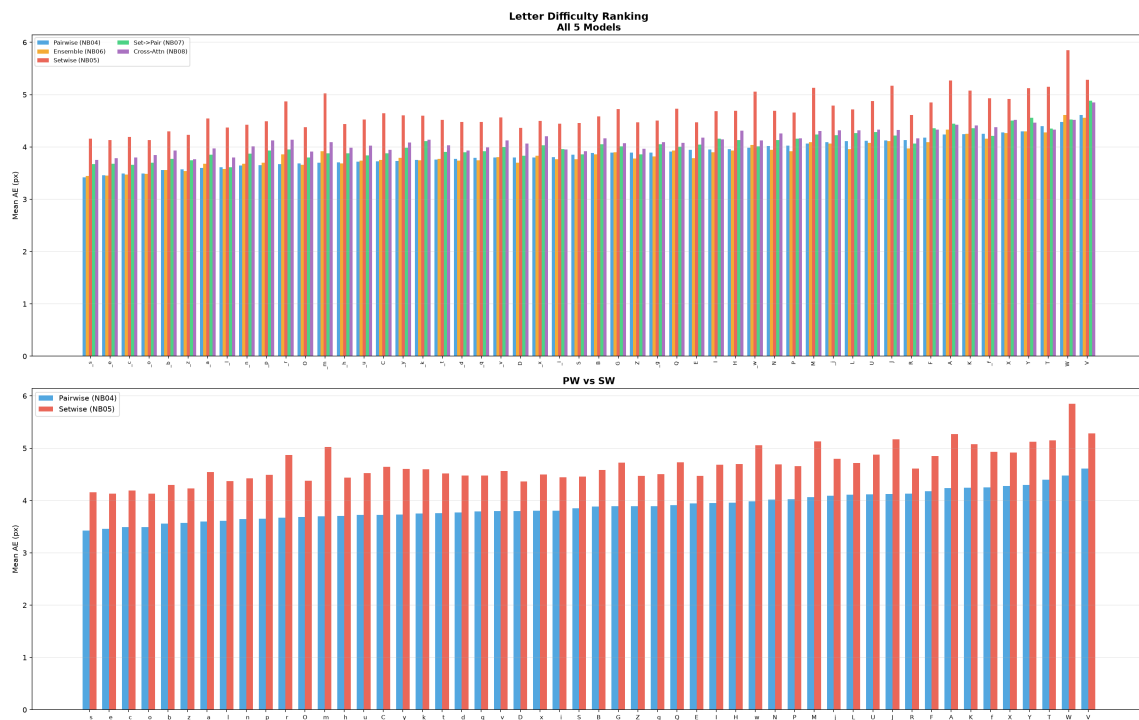


Figure 18: Letter difficulty ranking across the five models. The same letters occupy the hard end for every architecture.

Figure 19 closes the analysis with a qualitative view: for each style and model it overlays the ground-truth glyph placement (black) with the model prediction (colored), on the best and the worst font. On the best fonts the two placements coincide almost perfectly; on the worst fonts—decorative Display and Handwriting faces—the prediction drifts visibly away from the ground truth, mirroring the quantitative residue of Figure 11.

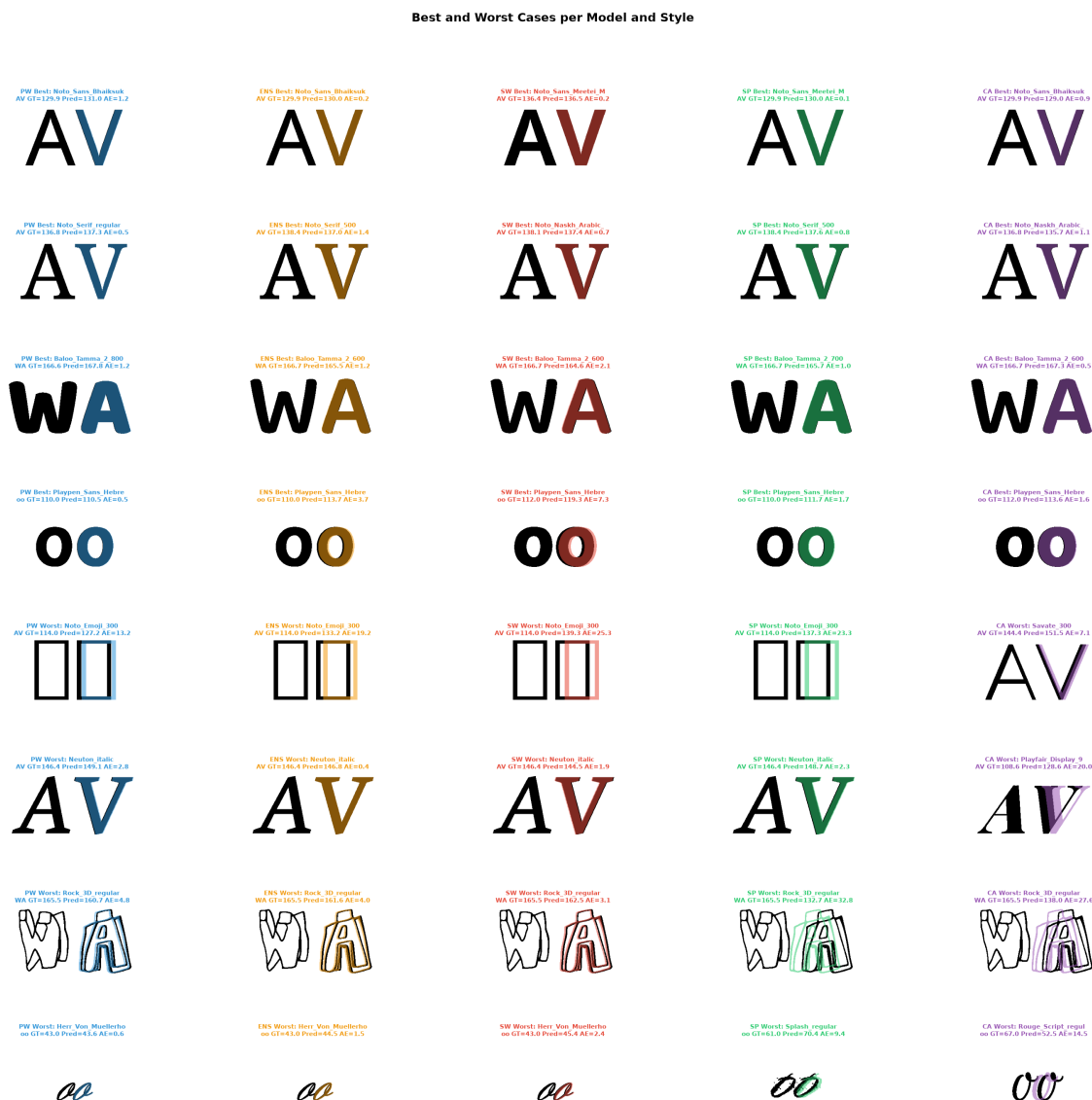


Figure 19: Best and worst cases per model and font style. Each cell overlays the ground-truth placement (black) and the model prediction (colored), making the residual spacing error visible.

Interactive Demo

To make the models inspectable, we built DLKern, a desktop application in Tkinter and Matplotlib. In *pair* mode the user selects a font and an ordered pair and sees the ground-truth rendering together with the renderings of any subset of the five models, followed by an overlay placing all predictions on the same pair. In *word* mode the user types a short text and the application spaces every adjacent pair under each selected model. Models load lazily, font features and set-wise hidden states are cached, and the inference layer is isolated in a standalone module so the checkpoints can be reused outside the notebooks. Figure 20 shows both modes: on regular text faces the pairwise and ensemble placements are visually indistinguishable from the ground truth, while the drift on decorative styles mirrors the quantitative results.

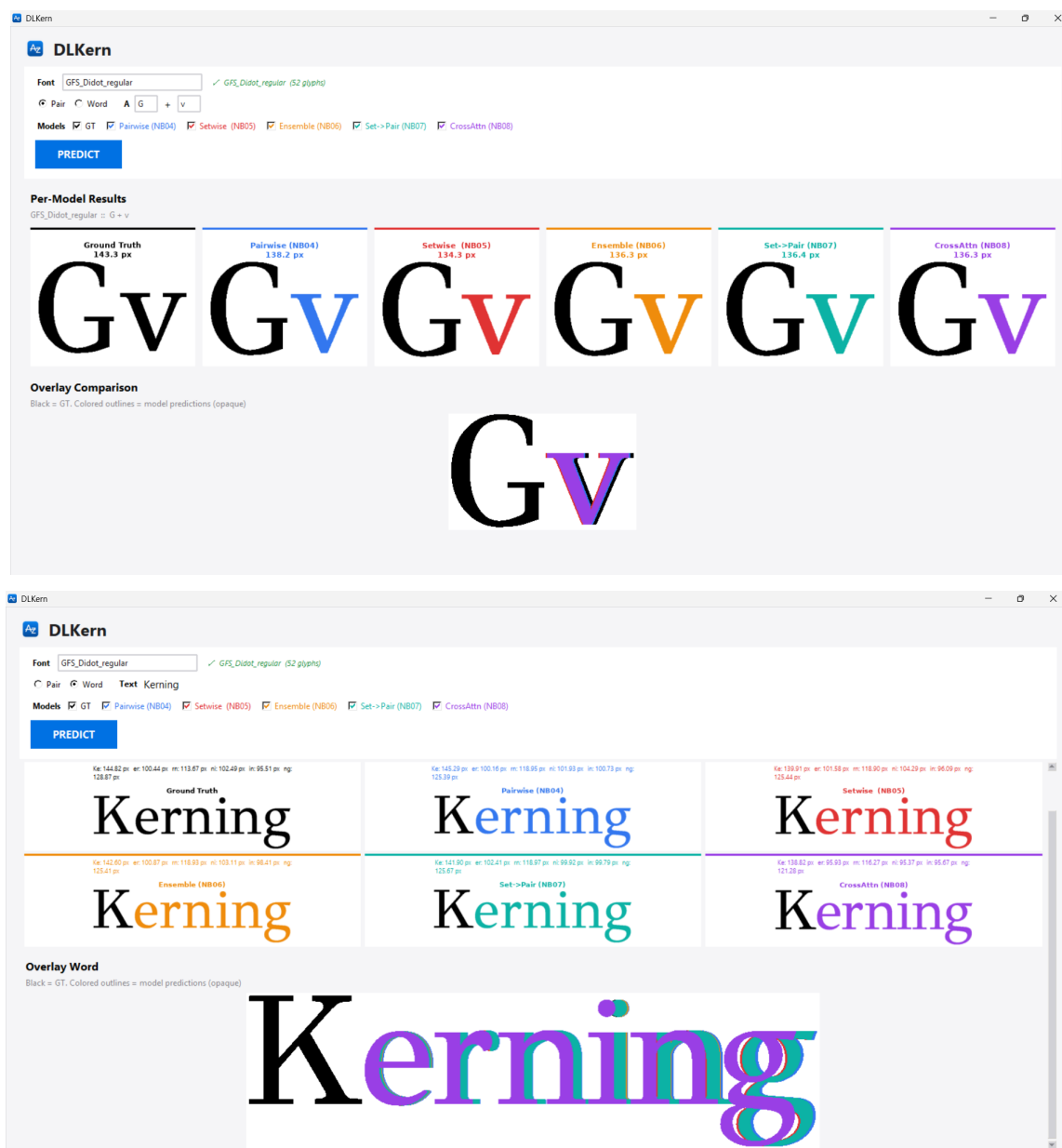


Figure 20: DLKern in pair mode (top, pair Gv) and word mode (bottom, Kerning), with ground truth and per-model overlays.

Conclusion and Future Work

We replicated and extended *Learning to Kern* on a corpus of 7,263 proportional fonts and 19.6 million all-pairs targets. The pairwise conditional model is the strongest single architecture at 3.89 px test MAE—70.7% below the reference pairwise result and about 3% of the mean spacing it regresses—and a training-free ensemble with the set-wise model is marginally better, its gain concentrated on the hardest fonts. The set-wise Transformer and both hybrids do not surpass the pairwise baseline: their attention is near-uniform, showing that a compact single-layer instantiation cannot exploit global structure over 2,704 tokens, and that hidden states frozen for one objective do not transfer to another. SHAP closes the loop: the pairwise model wins by regressing spacing from glyph shape, which is what the task mostly is once supervision is dense.

Three design statements generalize beyond this setting. First, target specification is an architectural decision: because renderers treat absent kerning as zero spacing, the regression target must enumerate every pair, and sparser supervision underconstrains the model. Second, at this scale data matters more than architectural sophistication: a small MLP trained on 5,758 fonts outperforms a Transformer trained on 2,046 in the reference study, reversing its internal ranking. Third, frozen representations are not automatically transferable: a representation optimized for one scalar loss is not a queryable feature space for another model.

The results are bounded by a single training seed without confidence intervals; the FontForge and reference baselines are cited from the original study, not re-executed; and the character set is fixed to 52 Latin letters with a frozen backbone, so the set-wise conclusion is conditional on its compact reference implementation. Natural next steps are a style-conditional ensemble (the ensemble and fusion models already win on Display and Handwriting respectively), larger set-wise models to give the global-context hypothesis a fair test, and extension to variable-length character sets and vector font formats, ideally with subjective evaluation by type designers.

The complete pipeline is released as a sequence of notebooks, from font download to explainability; trained checkpoints for the backbone and all five models are distributed, and the DLKern demo runs on standard hardware without retraining. The full project is available at <https://github.com/AngeloTetro/DL-Kerning>. All experiments use a fixed seed and the family-level split of Table 2, shared verbatim by every model.

References

- [1] Google. Google Fonts – Developer API Documentation, 2026. <https://developers.google.com/fonts>.
- [2] Kaiming He, Xiangyu Zhang, Shaoqing Ren, and Jian Sun. Deep Residual Learning for Image Recognition. *CVPR*, 2016. <https://arxiv.org/abs/1512.03385>.
- [3] Peter Karow. Two Decades of Typographic Research at URW: A Retrospective. *Electronic Publishing, Artistic Imaging, and Digital Typography*, 1998. <https://link.springer.com/chapter/10.1007/BFb0053276>.
- [4] David Kindersley. *Optical Letter Spacing for New Printing Systems*. Lund Humphries, 1976.
- [5] Diederik P. Kingma and Jimmy Ba. Adam: A Method for Stochastic Optimization. *ICLR*, 2015. <https://arxiv.org/abs/1412.6980>.
- [6] Scott M. Lundberg and Su-In Lee. A Unified Approach to Interpreting Model Predictions. *NeurIPS*, 2017. <https://arxiv.org/abs/1705.07874>.
- [7] Kei Nakatsuru and Seiichi Uchida. Learning to Kern: Set-wise Estimation of Optimal Letter Space. *arXiv preprint arXiv:2402.14313*, 2024. <https://arxiv.org/abs/2402.14313>.
- [8] FontForge project. FontForge, 2026. <https://fontforge.org>.
- [9] Ramprasaath R. Selvaraju, Michael Cogswell, Abhishek Das, et al. Grad-CAM: Visual Explanations from Deep Networks via Gradient-Based Localization. *ICCV*, 2017. <https://arxiv.org/abs/1610.02391>.
- [10] Laurens van der Maaten and Geoffrey Hinton. Visualizing Data using t-SNE. *Journal of Machine Learning Research*, 2008. <https://www.jmlr.org/papers/v9/vandermaaten08a.html>.
- [11] Ashish Vaswani, Noam Shazeer, Niki Parmar, et al. Attention Is All You Need. *NeurIPS*, 2017. <https://arxiv.org/abs/1706.03762>.